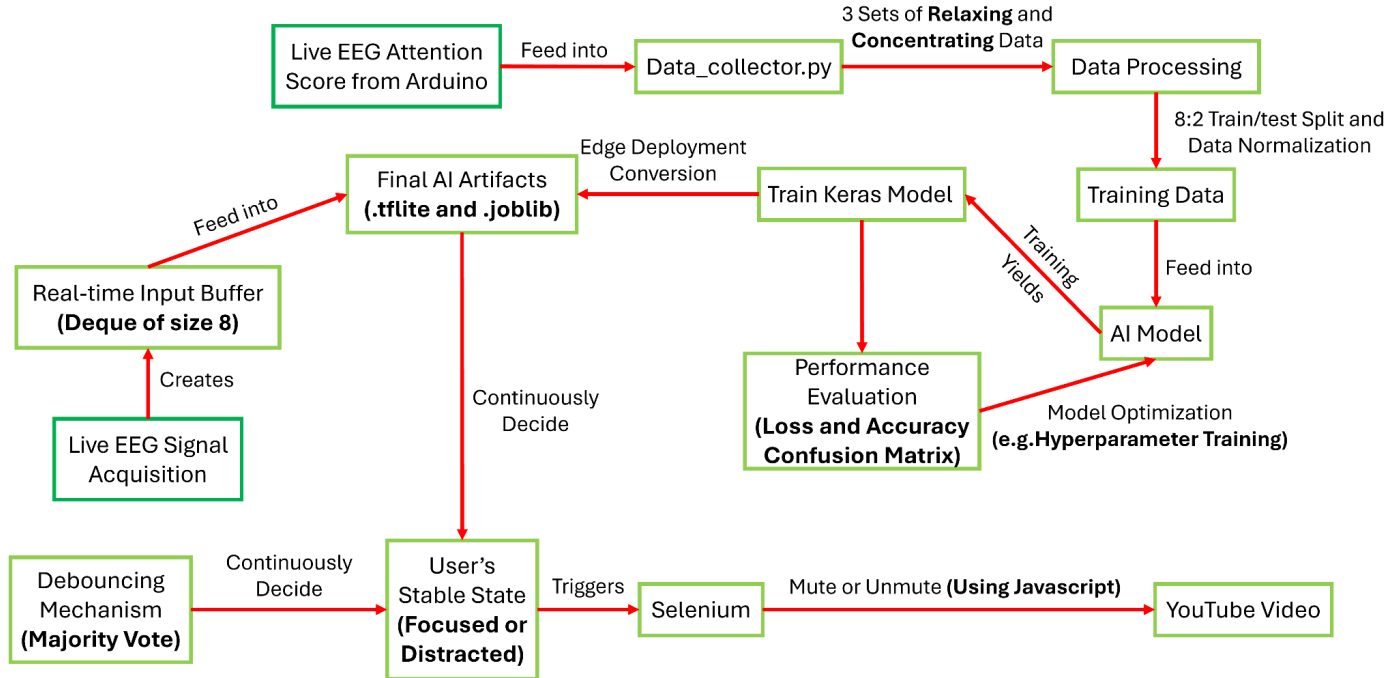


BMEG3103 Neuro-AI Project Summary

Group members: Cheung Wan Fung (SID: 1155175377)
Chen Yuk San (SID: 1155212369)
Ponchalee Tanaban (SID: 1155168418)

Target Application: Distraction Lock – browser tab auto-mutes YouTube when focus drops.

1.1 Simple Diagram (End-to-End System Architecture)



1.2 AI model justification / training process / accuracy

For model choice, a 1D Convolutional Neural Network (CNN) was adopted to learn temporal patterns from the sequences of incoming EEG attention scores. The network architecture was specifically designed to be both lightweight and effective for this time-series classification task. It is detailed as follows:

1. Hierarchical Feature Extraction: The model began with a stack of two sequential **Conv1D layers**.

The **first Conv1D layer**, with 8 filters and a kernel size of 3, extracted the initial feature of the input data. It slid across the 8-point window of recent attention scores, learning to recognize 8 different types of simple, low-level brainwave trends. For example, one filter may specialize in identifying “sudden drop in focus” (e.g. with scores going 88, 50, 20), while another specialize in recognizing “stable, low-attention state” (e.g. with scores of 10, 14, 11).

The **second Conv1D layer**, with 16 filters, then analyzed the output from the first layer and tried to learn more complex, hierarchical features by identifying combinations of the simpler trends. For example, it might learn that “stable high-focus” then “sudden drop” trend likely indicates “distraction” state.

Both layers used **ReLU (Rectified Linear Unit) activation function** to introduce non-linearity for learning complex relationships between different brainwave patterns, rather than just simple linear correlations.

2. Dimensionality Reduction & Feature Extraction:

A **MaxPooling1D layer** with pool size of 2 was applied after the convolutional stack, downsampling the feature map by half, retaining only the most important features. It makes our model more computationally efficient and prevents overfitting issues on the training data by reducing the number of parameters that need to be learned.

3. Classification Head:

A **Flatten layer** was used to flatten the multi-dimensional feature maps from the **MaxPooling1D layer** into a single, one-dimensional vector, which was then fed into a **Dense layer** containing a single neuron that employed a **sigmoid activation function**. This function outputs a value between 0.0 to 1.0 that can be directly interpreted as the predicted probability that the input data sequence belongs to the “Concentrating” class.

The **training process** consisted of several steps.

1. Data Collection & Cleaning

In **data_collector.py**, clean and labelled 60-second sessions for “relaxed” and “concentrating” class. Artefact handling is also implemented, which checks **SignalQuality** from the EEG sensor for each data point. Any readings with poor signal quality were automatically discarded to ensure that only clean and reliable data was saved for training the model.

2. Preprocessing & Feature Engineering

Data from all of our group members (3 in total) was aggregated into a single dataset, which then a sliding window of size 8 was applied to the time-series data to produce a series of short, overlapping sequences. It was then split into 80% training set and 20% hold-out testing set, subsequently underwent normalization (with a range between 0 and 1) using **MinMaxScaler**.

3. Model Training

Our model adopted **Adam optimizer** and **binary cross entropy loss function**, which is standard for binary classification tasks. To prevent overfitting issue, **Early Stopping** was employed to automatically stop the training process when the performance stops improving, restoring the model weights from the best-performing epoch. (which stopped at **22nd epoch** in our actual training process)

4. Edge Deployment

Upon completion of training, the final model (.h5) was converted and optimized into a lightweight (.tflite) file

For **accuracy & performance**, our model achieved an accuracy of **97.14%**, surpassing the project's **85% goal**.

The **classification report** and **confusion matrix** below confirm the model's high performance across both classes, with **strong F1-scores** indicating a good balance between **precision** and **recall**.

```
--- Model Performance Report ---
Accuracy on Hold-out Data: 97.14%
3/3 0s 12ms/step
```

Classification Report:				
	precision	recall	f1-score	support
Relaxed	0.95	1.00	0.97	35
Concentrating	1.00	0.94	0.97	35
accuracy			0.97	70
macro avg	0.97	0.97	0.97	70
weighted avg	0.97	0.97	0.97	70

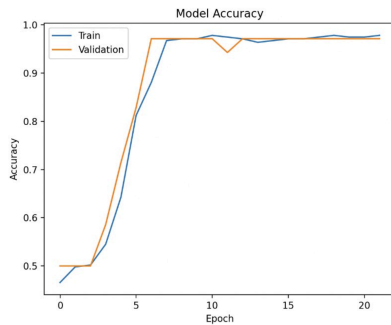


Fig 1. Classification Report

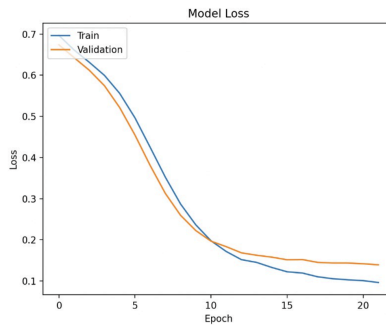


Fig 2. Model Accuracy & Loss across epochs

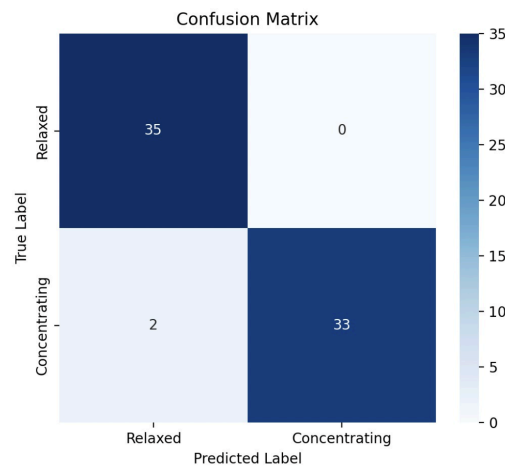


Fig 3. Confusion Matrix

1.3 Challenges

During this project, we encountered and successfully overcame several significant technical challenges that are common in real-world AI implementations.

1. YouTube Anti-Bot System Blockage

Our initial live demo attempt was blocked by YouTube's anti-bot systems, in which the automated chrome browser was flagged as "bot", causing "Something went wrong..." error.

Proposed Solution: We engineered a multi-layered "stealth" solution. The browser was launched with a special flag (`--disable-blink-features=AutomationControlled`) to hide its automated nature. We also used direct JavaScript commands (`driver.execute_script`) to control the player instead of simulating keystrokes, which bypassed the website's event listeners, thereby solving the problem.

2. Real-Time Control Instability

The raw output of the AI model, despite being accurate overall, would still fluctuate from moment to moment. As a result, the YouTube video rapidly and erratically mute and unmute (i.e., flickering effect occurred), leading to bad user experience.

Proposed Solution: We implemented a debouncing filter to smooth the output commands. Instead of acting on a single, instantaneous prediction, the script stores the last 5 predictions in a buffer. An action is only triggered when a majority vote confirms a stable change in the user's state (e.g., at least 3 of the last 5 predictions are "DISTRACTED"). This ensures the system's response is smooth, intentional, and human-like.